

DAY 4 | PostgreSQL 18 backup and recovery

Updated 28 September 2026. Run every command in your Ubuntu Bash terminal. Commands are plain selectable text. Paste one code block at a time. The commands use your actual namespace postgres-ha, standalone pod backup-lab-0 and Patroni cluster acid-postgres-ha.

Read this first: two separate databases

The pgBackRest repository /repo in backup-lab-0 contains backups of its own standalone PostgreSQL database, stanza lab. The three acid-postgres-ha pods are a different Patroni cluster. Backups from stanza lab cannot recover acid-postgres-ha. The standalone repository is on rancher.io/local-path storage on one Kubernetes node; it is a training backup, not protection against loss of that node.

Your GCP organization enforces constraints/iam.disableServiceAccountKeyCreation. Do not run gcloud iam service-accounts keys create, and do not configure pgBackRest with a JSON key that does not exist. A later GCS design needs a configured, verified workload identity for this non-GKE Kubernetes cluster. No GCS backup of acid-postgres-ha is claimed by this manual.

0. Check your environment

```
kubectl -n postgres-ha get pods -o wide
kubectl -n postgres-ha get pvc
kubectl -n postgres-ha get configmap backup-lab-config
kubectl -n postgres-ha exec backup-lab-0 -- gosu postgres pgbackrest --stanza=lab info
```

Expected: backup-lab-0 and acid-postgres-ha-0/1/2 are Running. The lab stanza says status: ok and shows your existing full, differential and incremental backup labels. If a command fails, stop at that command; do not guess a new stanza or repository path.

Command anatomy

kubectl exec runs inside the named pod. The first -- ends kubectl options. gosu postgres runs the following program as the postgres Linux user, which pgBackRest requires. Do not add a second -- after postgres. pgbackrest --stanza=lab selects the standalone lab configuration.

Lab 1 | Backup existing standalone database

1. Review existing backups

```
kubectl -n postgres-ha exec backup-lab-0 -- gosu postgres pgbackrest --stanza=lab info
```

You already created a full backup on 26 September 2026 and successfully ran an incremental backup on 28 September 2026. An existing backup chain is sufficient for the next exercise. Repeating a full backup is optional for a fresh run.

2. Create a fresh full backup (optional)

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=full backup
```

Expected: backup command end: completed successfully. Retention may expire older sets according to backup-lab-config; review info before choosing a restore set.

3. Make a differential backup

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=diff backup
```

A differential backup contains changed files since the latest full backup.

4. Make an incremental backup

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab --type=incr backup
```

An incremental backup contains changed files since the previous successful backup. Your 28 September command already finished successfully with label 20260926-161449F_20260928-174132I.

5. Verify the backup chain and WAL archive

```
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab info
kubectl -n postgres-ha exec backup-lab-0 -- \
  gosu postgres pgbackrest --stanza=lab check
```

Expected: stanza status ok; backup list includes a full and newer differential and incremental labels; check completes successfully. Do not rely on the backup command alone as a restore test.

Lab 2 | Safe isolated physical restore

This exercise creates a NEW PVC called day4-lab-restore-data. The repository backup-lab-repo is mounted read-only in a second pod, on the same node. Never mount pgdata-acid-postgres-ha-0/1/2 or data-backup-lab-0 as the restore target. Run on your single-node local-path training cluster. If the pod cannot mount both volumes, stop; do not change node placement or PVC names to force it.

1. Verify configuration and discover image and node

```
kubectl -n postgres-ha get configmap backup-lab-config \
-o jsonpath='{.data.pgbackrest.conf}'
kubectl -n postgres-ha get pod backup-lab-0 \
-o jsonpath='{.spec.containers[?(@.name=="postgres")].image}{"\n"}'
kubectl -n postgres-ha exec backup-lab-0 -- id postgres
```

Gate: ConfigMap output must contain [lab], repo1-path=/repo, and pg1-path. The image must be nonempty and contain pgBackRest and PostgreSQL 18. Note the uid and gid from id postgres. The next block reads them automatically. If the ConfigMap key is different, stop and inspect its keys rather than applying the manifest below.

2. Create a new empty recovery PVC

```
cat > day4-lab-restore-pvc.yaml <<'YAML'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: day4-lab-restore-data
  namespace: postgres-ha
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 3Gi
YAML
kubectl apply -f day4-lab-restore-pvc.yaml
kubectl -n postgres-ha get pvc day4-lab-restore-data
```

WaitForFirstConsumer storage may keep this PVC Pending until the toolbox pod is scheduled; this is normal. Make this PVC larger if your actual source database no longer fits in 3 GiB.

3. Prepare the isolated pod manifest

```
LAB_IMAGE=$(kubectl -n postgres-ha get pod backup-lab-0 \
-o jsonpath='{.spec.containers[?(@.name=="postgres")].image}')
LAB_NODE=$(kubectl -n postgres-ha get pod backup-lab-0 -o jsonpath='{.spec.nodeName}')
LAB_HOST=$(kubectl get node "$LAB_NODE" \
-o jsonpath='{.metadata.labels.kubernetes.io/hostname}')
LAB_UID=$(kubectl -n postgres-ha exec backup-lab-0 -- id -u postgres)
LAB_GID=$(kubectl -n postgres-ha exec backup-lab-0 -- id -g postgres)
test -n "$LAB_IMAGE" && test -n "$LAB_HOST" && test -n "$LAB_UID" && test -n "$LAB_GID"
printf 'image=%s node=%s uid=%s gid=%s\n' \
"$LAB_IMAGE" "$LAB_HOST" "$LAB_UID" "$LAB_GID"
```

Lab 2 | Isolated restore (continued)

The next heredoc writes a YAML file in the current directory. The recovery container sleeps until you issue restore commands. Its image is the same image used by backup-lab-0.

```
cat > day4-lab-restore-pod.yaml <<YAML
apiVersion: v1
kind: Pod
metadata:
  name: day4-lab-restore
  namespace: postgres-ha
spec:
  restartPolicy: Never
  nodeSelector:
    kubernetes.io/hostname: "$LAB_HOST"
  initContainers:
    - name: prepare-directory
      image: "$LAB_IMAGE"
      command: ["sh", "-ec"]
      args: ["mkdir -p /recovery/data; chown -R $LAB_UID:$LAB_GID /recovery/data"]
      securityContext:
        runAsUser: 0
      volumeMounts:
        - name: target
          mountPath: /recovery
  containers:
    - name: toolbox
      image: "$LAB_IMAGE"
      command: ["sh", "-c", "sleep infinity"]
      volumeMounts:
        - name: target
          mountPath: /recovery
        - name: repo
          mountPath: /repo
          readOnly: true
        - name: config
          mountPath: /etc/pgbackrest/pgbackrest.conf
          subPath: pgbackrest.conf
          readOnly: true
  volumes:
    - name: target
      persistentVolumeClaim:
        claimName: day4-lab-restore-data
    - name: repo
      persistentVolumeClaim:
        claimName: backup-lab-repo
    - name: config
      configMap:
        name: backup-lab-config
YAML
kubectl apply -f day4-lab-restore-pod.yaml
kubectl -n postgres-ha wait --for=condition=Ready pod/day4-lab-restore --timeout=180s
```

If wait times out, inspect `kubectl -n postgres-ha describe pod day4-lab-restore` and both PVCs; do not run restore until Ready.

4. Confirm the recovery path is empty and repository readable

```
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  sh -c 'test ! -e /recovery/data/PG_VERSION'
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres pgbackrest --stanza=lab info
```

Lab 2 | Restore and verify PostgreSQL starts

5. Restore the most recent valid backup set

```
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres pgbackrest --stanza=lab \
  --pg1-path=/recovery/data restore
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  cat /recovery/data/PG_VERSION
```

Expected: restore command end: completed successfully; PG_VERSION shows 18. Restoring the latest set can require earlier full/differential/incremental sets and WAL. PG_VERSION alone does not prove recovery.

6. Start the isolated restored database and run SQL

```
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres pg_ctl -D /recovery/data \
  -o "-c listen_addresses='' -c port=5544 -c unix_socket_directories=/tmp" \
  -l /recovery/data/day4-recovery.log -w start
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres psql -h /tmp -p 5544 -U postgres \
  -d postgres -c 'SELECT current_database(), pg_is_in_recovery();'
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres psql -h /tmp -p 5544 -U postgres \
  -d postgres -c 'SELECT datname FROM pg_database ORDER BY datname;'
```

Expected: SQL connects; pg_is_in_recovery() should become false after normal recovery. This server listens only on a Unix socket inside the disposable pod. If startup fails, inspect the log below and stop; do not point the operator at the recovery PVC.

```
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  sh -c 'tail -80 /recovery/data/day4-recovery.log'
```

7. Stop the isolated server when finished

```
kubectl -n postgres-ha exec day4-lab-restore -c toolbox -- \
  gosu postgres pg_ctl -D /recovery/data -m fast -w stop
```

Keep day4-lab-restore-data while reviewing your results. Deleting a local-path PVC can delete its recovery data. The source backup-lab-0 and Patroni database pods are not restarted by this exercise.

Lab 3 | Live Patroni backup status and next design

1. Inspect the actual live cluster

```
kubectl -n postgres-ha get pods -o wide
kubectl -n postgres-ha get pvc
kubectl -n postgres-ha get postgresql acid-postgres-ha \
-o jsonpath='{.spec.dockerImage}{"\n"}'
```

Expected: all three pods Ready. The configured image in your dry run is `docker.io/eswaribala/ericpostgrrepo2026:pg18`. Zalando generates a StatefulSet with update strategy `OnDelete` here; `kubectl rollout status` does not validate that strategy. Inspect actual pod image and readiness instead.

```
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  echo "$pod"
  kubectl -n postgres-ha get pod "$pod" \
  -o jsonpath='{.spec.containers[0].image}{"\n"}'
  kubectl -n postgres-ha get pod "$pod" \
  -o jsonpath='{.status.containerStatuses[0].ready}{"\n"}'
done
```

2. Understand what is still required for live physical backups

Before claiming protection of `acid-postgres-ha`, provision a separate durable repository accessible to the Patroni pods, configure `pgBackRest` on the actual primary and replicas, enable and verify WAL archiving, take a full backup, run `pgbackrest check/info` for that stanza, and restore to an isolated environment. The `local-path backup-lab-repo` is owned by the standalone lab and is not a failover-safe shared repository for the live three-pod cluster.

Your non-GKE Kubernetes cluster needs an approved cloud identity mechanism if you select GCS. A bucket existing in GCP or a Google login on the Ubuntu host does not authenticate Kubernetes pods. With service-account key creation blocked, use approved Workload Identity Federation or another repository architecture after confirming support for the actual `pgBackRest` image and operator configuration. No unverified `gcloud` key or custom image rebuild instructions are included here.

Optional logical backup of the live banking database

Only run after confirming which pod is currently primary and that your PostgreSQL role can connect locally; the example checks each pod using SQL, then saves a custom-format dump on the Ubuntu computer.

```
mkdir -p backups/logical
for pod in acid-postgres-ha-0 acid-postgres-ha-1 acid-postgres-ha-2; do
  kubectl -n postgres-ha exec "$pod" -c postgres -- \
  psql -U postgres -d postgres -Atqc 'SELECT pg_is_in_recovery()' \
  && echo "pod=$pod"
done
```

Select the pod that prints `f` as PRIMARY. Set it exactly to that pod name; do not assume pod 0 is primary. If authentication fails, stop and use a validated credential method for your cluster.

```
PRIMARY=acid-postgres-ha-0
kubectl -n postgres-ha exec "$PRIMARY" -c postgres -- \
  pg_dump -U postgres -Fc banking > backups/logical/banking.dump
test -s backups/logical/banking.dump && ls -lh backups/logical/banking.dump
```

Replace the example PRIMARY assignment with the pod that printed `f`. This logical file is local to Ubuntu, separate from `pgBackRest` and the `backup-lab` repository. Do not claim PITR protection from `pg_dump` alone.

Reference, troubleshooting and completion checks

What each backup covers

lab full/diff/incr: backup-lab-0 database and WAL in its local PVC repository. banking.dump: one live Patroni database at dump time on your Ubuntu computer. Neither is a verified live-cluster physical backup with off-node disaster recovery.

Common failures

ERROR [031] pgBackRest must not run as root: use gosu postgres. kubectl says executable '--' not found: remove the extra -- after postgres. kubectl edit cancelled: no saved CR change; check spec.dockerImage and edit again. kubectl rollout status unsupported: check OnDelete pod readiness and actual images instead. GCS FAILED_PRECONDITION: service account key creation blocked by organization policy; stop that path. PVC Pending: WaitForFirstConsumer may wait for scheduling; inspect pod events. PG_VERSION exists in a recovery target: do not rerun restore on it; use a new isolated PVC after completing the current drill.

Lab success checklist

Standalone: pgbackrest info status ok; full/diff/incr visible; check successful; isolated recovery PVC separate from all existing database PVCs; restored PostgreSQL 18 starts; SQL query succeeds. Live Patroni: three pods Ready and their images verified; a logical banking.dump is nonempty if you ran the optional logical exercise. Live physical backup and PITR remain a separate exercise requiring a supported durable repository and verified identity.

Sources and scope

Official pgBackRest user guide: <https://pgbackrest.org/user-guide.html>

Google Cloud IAM organization policy and Workload Identity Federation: <https://cloud.google.com/iam/docs/service-account-creds> and <https://cloud.google.com/iam/docs/workload-identity-federation>

Commands in this manual were written for the environment shown in your screenshots. Execution against your cluster and the isolated restore outcome must still be verified there.